

Eliminate False Sharing

Stop your CPU power from invisibly going down the drain

By Herb Sutter

Herb Sutter is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca.

In two previous articles I pointed out the performance issue of false sharing (aka cache line ping-ponging), where threads use different objects but those objects happen to be close enough in memory that they fall on the same cache line, and the cache system treats them as a single lump that is effectively protected by a hardware write lock that only one core can hold at a time. [1,2] This causes real but invisible performance contention; whichever thread currently has exclusive ownership so that it can physically perform an update to the cache line will silently throttle other threads that are trying to use different (but, alas, nearby) data that sits on the same line. It's easy to see why the problem arises when multiple cores are writing to different parts of the same cache line, because only one can hold the exclusive hardware lock at a time. In practice, however, it can be even more common to encounter a reader thread using what it thinks is read-only data still getting throttled by a writer thread updating a different but nearby memory location, because the reading thread has to invalidate its copy of the cache line and wait until after the writer has finished to reload it.

A number of readers have asked for more information and examples on where false sharing arises and how to deal with it. I mentioned one concrete example in passing in [3] where Example 4 showed eliminating false sharing as one of the stages of optimizing a queue.

This month, let's consider a concrete example that shows an algorithm in extremis due to false sharing distress, how to use tools to analyze the problem, and the two coding techniques we can use to eliminate false sharing trouble.

The Little Parallel Counter That Couldn't

Consider this sequential code to count the number of odd numbers in a matrix:

```
int odds = 0;
for( int i = 0; i < DIM; ++i )
    for( int j = 0; j < DIM; ++j )
        if( matrix[i*DIM + j] % 2 != 0 )
            ++odds;
```

If our job is to parallelize existing code, this is just what the doctor ordered: An embarrassingly parallel problem where it should be trivial to achieve linear speedups simply by assigning $1/P$ -th of the independent workload to each of P parallel workers. Here's a simple way to do it:

```
// Example 1: Simple parallel version (flawed)
//
int result[P];
```

```

// Each of P parallel workers processes 1/P-th
// of the data; the p-th worker records its
// partial count in result[p]
for( int p = 0; p < P; ++p )
    pool.run( [&,p] {
        result[p] = 0;
        int chunkSize = DIM/P + 1;
        int myStart = p * chunkSize;
        int myEnd = min( myStart+chunkSize, DIM );
        for( int i = myStart; i < myEnd; ++i )
            for( int j = 0; j < DIM; ++j )
                if( matrix[i*DIM + j] % 2 != 0 )
                    ++result[p];
    } );
// Wait for the parallel work to complete
pool.join();
// Finally, do the sequential "reduction" step
// to combine the results
odds = 0;
for( int p = 0; p < P; ++p )
    odds += result[p];

```

Quick: How well would you expect Example 1 to scale as P increases from 1 to the available hardware parallelism on the machine? You already have a hint from the topic of this column -- is there any part of the code that might worry you?

Let's find out. When I ran the code in Example 1 with values of P from 1 to 24 on a 24-core machine, I got the results shown in Figure 1.

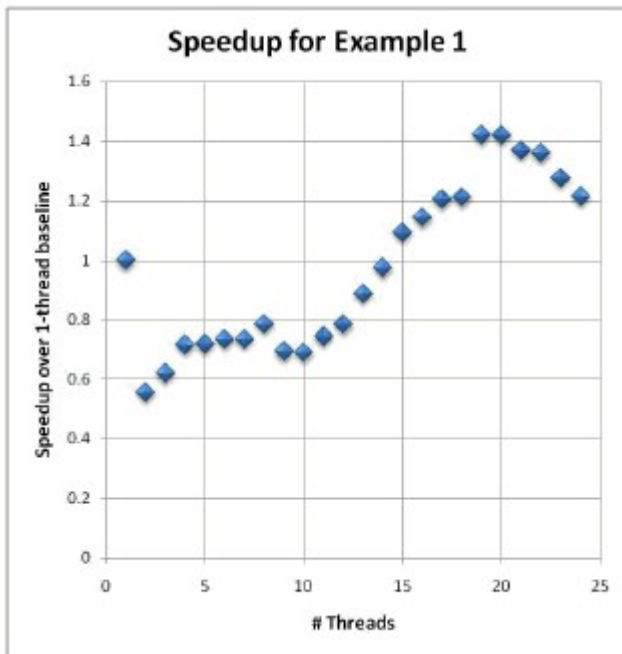


Figure 1: Example 1 seems to be about how to use more cores to get less total work done.

These results aren't just underwhelming; they're staggeringly bad. In most cases, the parallel code actually ran slower than the sequential code, and in no case did we get any better than a 42% speedup no matter how many cores we threw at the problem. Yet this is supposed to be an embarrassingly parallel (i.e., embarrassingly easy to scalably parallelize) algorithm suitable for an introductory concurrency class. What's going on?

Analyzing What Went Wrong

To figure out where the problem lies, perhaps the first thing you might try to do is run this code while watching a CPU monitor to see which system cores are busy and for how long. If we run the example code with P set to 1, which makes it run sequentially, then we would expect to see one hardware core light up for the duration of the execution. Figure 2 shows that this is in fact exactly what happened on my test system. [4]

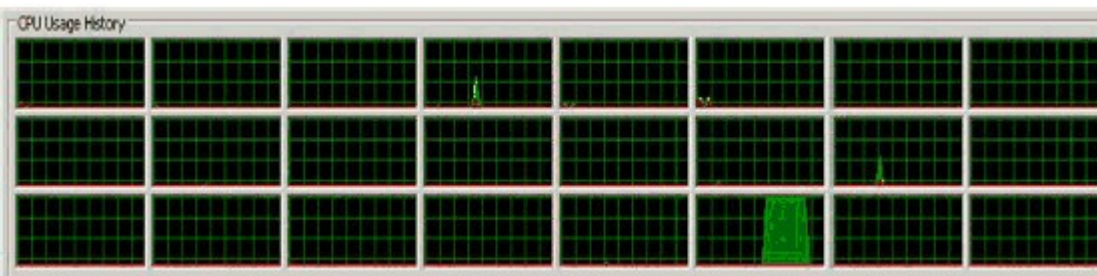


Figure 2: Running Example 1 with $P = 1$ (sequential baseline case).

Now let's run Example 1 again with $P = 14$, which will use 14 hardware cores if available, and watch the CPU monitor to see which cores are busy and for

how long. I picked 14 just because in Figure 1 the execution time for $P = 14$ was about the same as for $P = 1$, so we can compare CPU utilization for about the same execution time as well as the same workload; remember that regardless of the value of P every execution is doing exactly the same amount of counting work on the identical data set, just carving the work up P different ways. Figure 3 shows the result on my system.

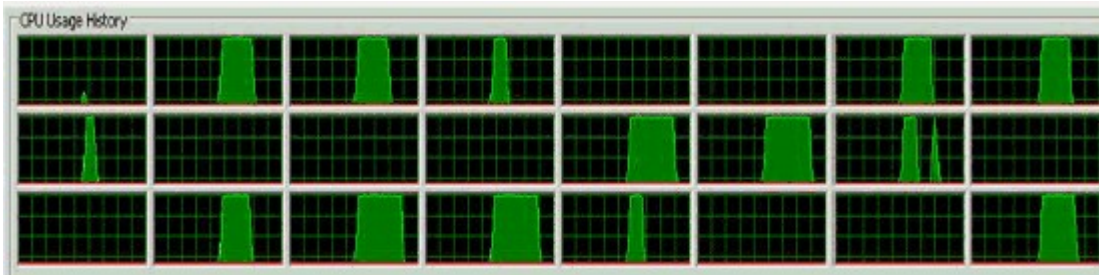


Figure 3: Running Example 1 with $P = 14$ (ugh, approximately equal execution time as for $P = 1$; see Figure 1).

What does Figure 3 tell us? First, it confirms that the parallel version is indeed lighting up 14 different cores, so we didn't make a mistake and accidentally run sequentially on a single core. So far, so good. Second, we see that some of the cores stay busy for as long as the single core in Figure 2 did, which is why the total execution time is about the same. Not great, to be sure, but at least it's consistent with our previous data.

Third, if you add up the total CPU busy time in Figure 3, we clearly used much more total CPU horsepower than in Figure 2. Yet both executions performed exactly the same number of traversals and additions; the work was merely divided differently. Now that's just crazy; it means the CPU monitor must somehow be lying to us, because it claims that the cores are pegged and busily computing when that can't be true because we know full well the total number of matrix cell visits and counter additions that our program is doing is identical as in the execution in Figure 2. Hmm. Well, for now, let's just remember this on our list of mysteries to be solved and press on.

Fourth, we find a new clue: We can see some of the cores didn't stay busy as long as others. Now that's a puzzling thing, because each of the 14 workers was given an identically sized chunk of work to do and so should have taken the same length of time to do it. Yet some workers apparently ran faster (or slower) than others. Hmm; add that to the mystery list, too.

That's about as far as we can go with a CPU monitor. Time for other tools.

If you run this code under a performance analyzer that lets you examine the cycles per instruction (CPI) for each line of source code, you'll probably discover that the CPI for one line in particular is amazingly high: The offending line is `++result[p]`. Now, that ought to strike us as strange, because `++result[p]` isn't some heavyweight function call; it's just computing an offset into an array and then incrementing an integer, and so should have a very low CPI.

Next, if you run the code under a performance analyzer that can measure cache misses, or better still cache line contention, and attribute the cost to particular lines of code, you'll get even more targeted information about the line `++result[p]`: It's a hot spot that's spending its time waiting for memory, specifically for cache line ownership.

Put the CPI and cache miss information together, and there we have it: A classic case of false sharing. Each worker is incrementing its own distinct counter, but the counter values are adjacent in the result array. To increment its counter, a worker must have exclusive ownership of the cache line containing the counter, which means that the other workers trying to update their counters elsewhere in that same cache line must stall and wait until they can in turn get exclusive access to the line containing their chunk of result. The ownership of the cache line ping-pongs madly from core to core, and only one core can be running at a time, silently throttling the life out of the program.

For reasons that are invisible in the code, we ended up writing, not a truly parallel algorithm, but just a complicated sequential algorithm.

From Zero to Hero: The Little Parallel Counter That Could

The simplest way to fix the problem is simply to have each p -th worker increment its own local variable, and only at the end write its final tally to **result[p]**. It's an amazingly small change to the code:

```
// Example 2: Simple parallel version
// (de-flawed using a local variable)
//
int result[P];

// Each of P parallel workers processes 1/P-th
// of the data; the p-th worker records its
// partial count in result[p]
for( int p = 0; p < P; ++p )
    pool.run( [&p] {
        int count = 0;
        int chunkSize = DIM/P + 1;
        int myStart = p * chunkSize;
        int myEnd = min( myStart+chunkSize, DIM );
        for( int i = myStart; i < myEnd; ++i )
            for( int j = 0; j < DIM; ++j )
                if( matrix[i*DIM + j] % 2 != 0 )
                    ++count;
        result[p] = count;
    } );
// etc. as before
```

Could such a small change really make a big difference in scalability? Let's measure and find out. When I ran the code in Example 2 with values of P from 1 to 24 (on a 24-core machine), I got the results shown in Figure 4.

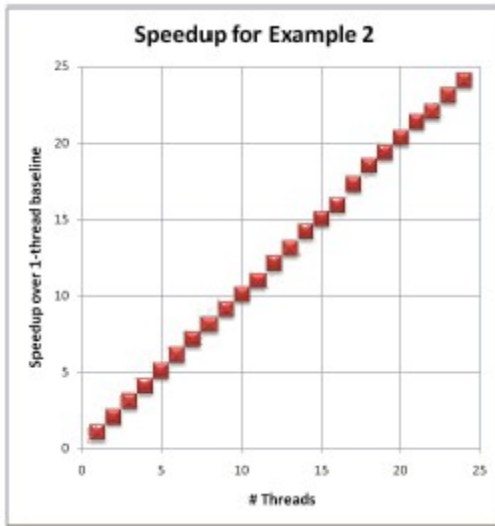


Figure 4: Removing cache line contention on the result array takes us from zero scaling to linear scaling, up to the available hardware parallelism (test run on a 24-core machine).

The amended code's scalability isn't just better -- it's perfect scaling, linear in the number of processors. Figure 5 shows the work per CPU core with $P = 24$; each core's work was complete so fast that it didn't manage to peg the core long enough to fill a whole CPU monitor polling interval.

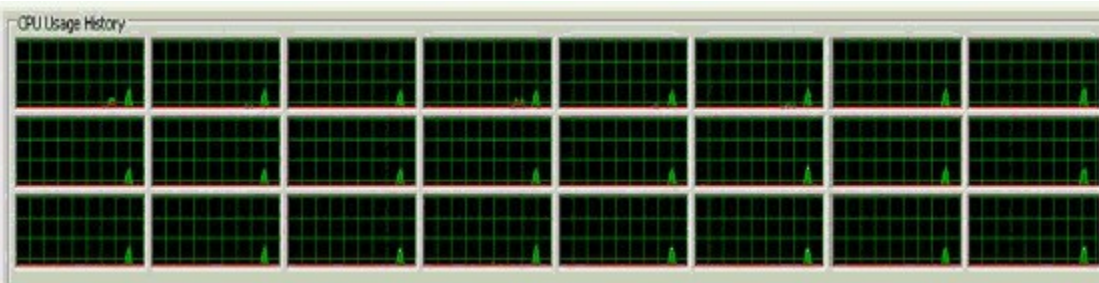


Figure 5: Running Example 2 with $P = 24$ — now that's more like it, the kind of workload distribution we want to see.

Now that we've confirmed the culprit in Example 1 was memory contention due to false sharing on the result array, this helps clear up the mystery of why the cores appeared pegged in the CPU monitor (Figure 3) when they were actually not doing as much work: Many CPU monitors, like this one, count the time a core is waiting for cache and memory as part of its "busy" time. After all, the core is executing an instruction; it just happens to be an expensive memory fetch instruction. That explains why a core can appear to be fully utilized when it's actually only doing useful computation work a fraction of the time; it's spending the rest of its time just waiting for memory.

We've also cleared up the mystery of why some workers finished faster than others in Figure 3: The ones that took longer were the ones that were experiencing more contention because their counters happened to be on cache lines containing a greater number of other workers' counters. Workers whose counters happened to be on less-popular cache lines had to wait less and so ran faster.

Our small code change has taken us from zero scaling to perfect scaling: Now that's a zero-to-hero technique worth knowing about.

False Sharing: What To Look For

Note that Example 1 shows only one common case where data can end up being close together in memory, namely the case of elements of contiguous arrays. But the same thing can occur when we have two independently used fields within the same popular object, or objects are close to each other on the heap, or other situations.

The general case to watch out for is when you have two objects or fields that are frequently accessed (either read or written) by different threads, at least one of the threads is doing writes, and the objects are so close in memory that they're on the same cache line because they are:

- objects nearby in the same array, as in Example 1 above;
- fields nearby in the same object, as in Example 4 of [3] where the head and tail pointers into the message queue had to be kept apart;
- objects allocated close together in time (C++, Java) or by the same thread (C#, Java), as in Example 4 of [3] where the underlying list nodes had to be kept apart to eliminate contention when threads used adjacent or head/tail nodes;
- static or global objects that the linker decided to lay out close together in memory;
- objects that become close in memory dynamically, as when during compacting garbage collection two objects can become adjacent in memory because intervening objects became garbage and were collected; or
- objects that for some other reason accidentally end up close together in memory.

What To Do

When two frequently-used objects are sources of false sharing because they're in the same far-too-popular cache line, there are two general ways to remove the false sharing.

First, we can reduce the number of writes to the cache line. For example, writer threads can write intermediate results to a scratch variable most of the time, then update the variable in the popular cache line only occasionally as needed. This is the approach we took in Example 2, where we changed the code to update a local variable frequently and write into the popular result array only once per worker to store its final count.

Second, we can separate the variables so that they aren't on the same cache line. Typically the easiest way to do this is to ensure an object has a cache line to itself that it doesn't share with any other data. To achieve that, you need to do two things:

- Ensure that no other object can precede your data in the same cache line by aligning it to begin at the start of the cache line or adding sufficient padding bytes before the object.
- Ensure that no other object can follow your data in the same cache line by adding sufficient padding bytes after the object to fill up the line.

Here's how you can write this as a reusable library component in C++:

```
// C++ (using C++0x alignment syntax)
template<typename T>
struct cache_line_storage {
    [[ align(CACHE_LINE_SIZE) ]] T data;
    char pad[ CACHE_LINE_SIZE > sizeof(T)
              ? CACHE_LINE_SIZE - sizeof(T)
              : 1 ];
};
```

To get an object of type **MyType** that is stored on its own cache line, we would write **cache_line_storage<MyType>**. Note that this code assumes you've defined **CACHE_LINE_SIZE** to a suitable value for your target processor, commonly a power of two from 16 to 512. It also uses the standardized C++ **+0x** alignment syntax; if you don't have that yet, you can use a compiler-specific extension like Gnu's **__attribute__((aligned(x)))** or Microsoft's **__declspec(align(x))**.

If you're on .NET, you can write something similar but for value types only, which in their unboxed form are always laid out "inline" rather than as a separate heap object:

```
// C#: Note works for value types only
//
[StructLayout(LayoutKind.Explicit, Size=2*CACHE_LINE_SIZE)]
public struct CacheLineStorage<T>
    where T : struct
{
    [FieldOffset(CACHE_LINE_SIZE)] public T data;
}
```

It may seem strange that this code actually allocates enough space for two cache lines' worth of data instead of just one. That's because, on .NET, you can't specify the alignment of data beyond some inherent 4-byte and 8-byte alignment guarantees, which aren't big enough for our purposes. Even if you could specify a starting alignment, the compacting garbage collector is likely to move your object and thus change its alignment dynamically. Without alignment to guarantee the starting address of the data, the only way to deal with this is to allocate enough space both before and after data to ensure that no other objects can share the cache line.

For Java and .NET full-fledged objects (reference types), the solution is basically the same as for .NET value types, but more intrusive: You need to add the before-and-after padding internally inside the object itself because there is no portable way to add external padding directly adjacent to an object.

Applying this second approach to Example 1, we could change just the definition of the result array to space the array elements far enough apart. For example:

```
// Example 3: Simple parallel version (de-flawed using padding)
//
cache_line_storage<int> result[P];
// etc. as before, just replacing result[p] with result[p].data
```

Running performance tests confirms that this results in the same scalability curve as Example 2.

Finally, Don't Forget This Affects Readers, Too

In Example 1 we've been considering the case where all workers are writers, but readers are affected too. Consider the following variant of Example 1 where we arbitrarily force half the workers to only perform reads from their **result[p]**, and measure the program's execution to see what happens:


```

// Example 3: Simple parallel version with half the accesses as
// reads (still flawed)
//
int result[P];
// Each of P parallel workers processes 1/P-th of the data;
// the p-th worker records its partial count in result[p]
for( int p = 0; p < P; ++p )
    pool.run( [&,p] {
        int local = 0;
        result[p] = 0;
        int chunkSize = DIM/P + 1;
        int myStart = p * chunkSize;
        int myEnd = min( myStart+chunkSize, DIM );
        for( int i = myStart; i < myEnd; ++i )
            for( int j = 0; j < DIM; ++j )
                if( matrix[i*DIM + j] % 2 != 0 )
                    if( p % 2 != 0 ) // arbitrarily have every second
                        // worker
                            local += result[p]; // only read from its
                                                    // unshared result[p]
                    else
                        ++result[p];
    } );
// etc. as before

```

How's the performance? To paraphrase Family Feud, in Figure 6 our "survey saaaays"

Alas, Example 3 is roughly just as bad as Example 1, and with more erratic performance behavior to boot. Even with half the workers performing only reads of their **result[p]**, even at $P = 23$ we get about the same performance as when $P = 1$.

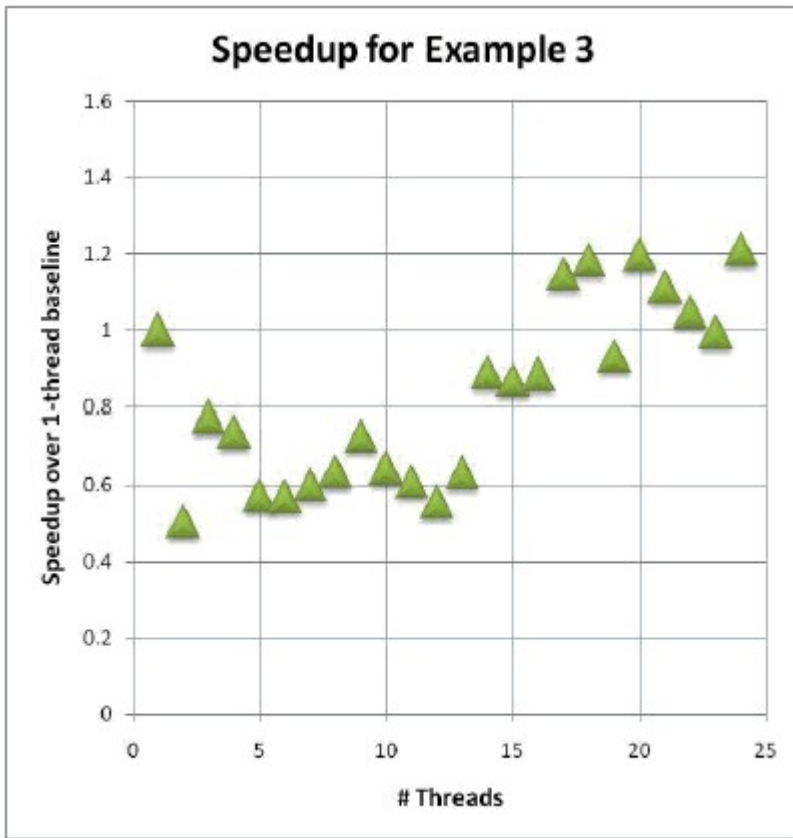


Figure 6: Example 3, with half the threads doing reads, is just as awful as Example 1.

Figure 7 shows the CPU monitor screen shots for $P = 1$ and $P = 23$, and confirms that for $P = 23$ we're lighting up 23 cores without any useful effect on total execution time.



Figure 7: Running Example 3 with $P = 1$ and $P = 23$ (ouch, approximately equal execution time to perform the same work, as we saw in Figure 6).

Finally, Figure 8 summarizes the relative scalability of Examples 1 to 3 side by side, where ideal linear scalability would be a horizontal line at scaling = 1. As we've already seen, Example 2 scales perfectly while Examples 1 and 3 don't scale at all -- and for no other reason than false sharing, even though in Example 3 half the workers are merely performing reads.

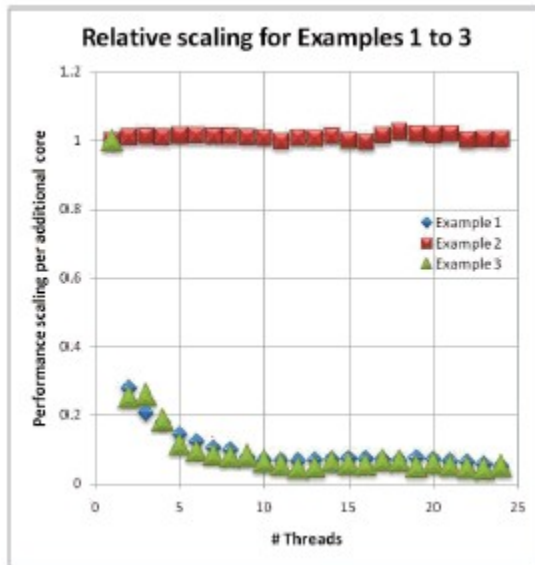


Figure 8: Examples 1 to 3 side by side (1 = perfect scaling).

Summary

Watch out for false sharing; it's an invisible scalability buster. The general case to watch out for is when you have two objects or fields that are frequently accessed (either read or written) by different threads, at least one of the threads is doing writes, and the objects are so close in memory that they're on the same cache line.

Detecting the problem isn't always easy. Typical CPU monitors completely mask memory waiting by counting it as busy time, which doesn't help us here, although the irregular lengths of the individual cores' busy times gives us a clue. Look for code performance analysis tools that let you measure, for each line of your source code, the cycles per instruction (CPI) and/or cache miss rates those source statements actually experience at execution time, so that you can find out which innocuous statements are taking extremely disproportionate amounts of cycles to run and/or spending a lot of time waiting for memory. You should never see high cache miss rates on a variable being updated by one thread in a tight inner loop, because it should just be loaded into cache once and then stay hot; lots of misses mean lots of contention on that variable or on a nearby one.

Resolve false sharing by reducing the frequency of updates to the falsely shared variables, such as by updating local data instead most of the time. Alternatively, you can ensure a variable is completely unshared by using padding, and alignment if available, to ensure that no other data precedes or follows a key object in the same cache line.

Acknowledgments

Thanks to Joe Duffy and Tim Harris for their observations and comments on this article.

Notes

[1] H. Sutter. ["Sharing Is the Root of All Contention" \(Dr. Dobb's Digest, March 2009\)](http://www.ddj.com/go-parallel/article/showArticle.jhtml?articleID=214100002). originally at <http://www.ddj.com/go-parallel/article/showArticle.jhtml?articleID=214100002>.

[2] H. Sutter. ["Maximize Locality, Minimize Contention." \(Dr. Dobb's Journal, 33\(6\), June 2008.\)](http://www.ddj.com/architect/208200273) originally at <http://www.ddj.com/architect/208200273>.

[3] H. Sutter ["Measuring Parallel Performance: Optimizing a Concurrent Queue" \(Dr. Dobb's Journal, 34\(1\), January 2009\)](http://www.ddj.com/cpp/211800538). originally at <http://www.ddj.com/cpp/211800538>.

[4] If you run this test yourself, you might instead see one core light up for a while, then go dark and a different core light up for a while, and so on. That's just an artifact of operating system thread scheduling, as the OS moves the thread to a different core for its own reasons, for example to keep heat distributed more evenly across the chip. The result is still logically equivalent to that in Figure 2 because only one core is running at a time, and the total execution time should not be materially affected by just occasional migration to a different core.

[Click here for more Dr. Dobbs articles from the archive!](#)